

# L'integrazione e la conoscenza (RAG)

Come si fa entrare la documentazione aziendale in una finestra di contesto piccola: dal contesto scritto a mano al recupero per similarità.

# Il problema: la conoscenza del modello è congelata al momento del training

- I pesi sono il risultato di un addestramento chiuso in una data: dopo quel momento non cambiano più.
- Usare il modello non lo aggiorna: nessuna conversazione entra nei pesi, come visto sullo stateless.
- Non ha connessioni: nessun accesso a database, file system o intranet se non gliele diamo noi.
- Aggiornare i pesi (fine-tuning) è un progetto a sé: costoso, lento, e da rifare a ogni cambio di dati.

DUE OROLOGI DIVERSI

**pesi** fermi alla data di training

**dati** cambiano ogni turno di produzione

Fra i due non c'è nessun canale automatico.

# Conseguenza pratica: non conosce i dati della tua azienda né eventi recenti

## DOMANDA

«Quante unità di MAT-4471/B abbiamo in magazzino?»

## RISPOSTA SENZA CONTESTO

«Risultano 128 unità disponibili nel deposito centrale.» — formato giusto, numero inventato.

- Il modello non distingue «non lo so» da «non è nei miei dati»: continua a prevedere il token successivo, come sempre.
- Fuori perimetro finiscono: procedure interne, giacenze, ordini, nomi di sistemi, personale, versioni di release.
- Sono gli errori peggiori da intercettare: linguaggio corretto, struttura corretta, contenuto falso.

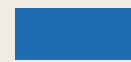
# Un'idea semplice: scrivere il contesto necessario dentro il prompt (in-context learning)

Contesto: MAT-4471/B – giacenza 37 pz, deposito B2, agg. 05/09/2026. Domanda: Quante unità di MAT-4471/B abbiamo in magazzino?

- Se il dato è nella finestra, il modello lo usa: la previsione del token successivo si appoggia al testo presente.
- «Learning» è un termine improprio: i pesi non cambiano, e finita la chiamata il dato non c'è più.
- Nessun addestramento, nessuna infrastruttura nuova: è solo testo aggiunto prima della domanda.
- Tutta la partita si sposta su una domanda: *quale* testo mettere lì.

# Il limite: non ci sta tutta la documentazione aziendale in un prompt

finestra del modello



8.192 token

manuale procedure (PDF)



~420.000 token

- Due ordini di grandezza di troppo: il corpus non entra, e non entra nemmeno con finestre molto più grandi.
- Più contesto inutile, meno precisione: l'informazione giusta si perde tra il rumore.
- Anche quando entra: si paga a token, a ogni turno, per testo in gran parte irrilevante.
- Serve quindi un passo prima della chiamata: selezionare i pochi frammenti che servono a *questa* domanda.

# La soluzione: cercare solo i pezzi rilevanti per similarità (gli embedding del modulo 2)

- Una volta sola: i documenti si spezzano in frammenti (chunk) di poche centinaia di token e si trasformano in vettori.
- I vettori stanno in un indice (vector store), con il testo originale e i metadati accanto.
- A ogni domanda: si vettorializza la domanda e si prendono i  $k$  chunk più vicini.
- Non è ricerca per parole chiave: «scorte esaurite» trova «giacenza a zero» anche senza termini in comune.

## TOP-K PER LA DOMANDA

|                      |             |      |
|----------------------|-------------|------|
| giacenze_B2.csv #14  | <div></div> | 0.83 |
| proc_riordino.md #3  | <div></div> | 0.64 |
| listino_2025.pdf #71 | <div></div> | 0.21 |

Con  $k = 2$  il terzo frammento resta fuori dal prompt.

# La pipeline RAG: cerca → recupera → genera solo su quella base

## 1 · CERCA

La domanda diventa un vettore e interroga l'indice. Nessun LLM coinvolto.

## 2 · RECUPERA

I chunk migliori vengono incollati nel prompt, con la loro origine.

## 3 · GENERA

Il modello risponde con l'istruzione di usare solo quel materiale.

```
system: "Rispondi solo usando i documenti forniti. Se non bastano, dillo." user: "<chunk 1> <chunk 2>\n\nDomanda: quante unità di MAT-4471/B?"
```

I primi due passi sono codice vostro: il modello resta stateless, cambia solo cosa gli arriva nella finestra.

# Perché riduce le allucinazioni: il modello cita, non inventa

- Il compito cambia: da «ricostruisci a memoria» a «riformula ciò che hai davanti» — molto più facile da azzeccare.
- Ogni frammento porta la sua origine: la risposta diventa verificabile riga per riga.
- Riduce, non elimina: se il recupero porta il documento sbagliato, il modello sarà convincente sul dato sbagliato.
- La qualità del sistema si sposta sul retrieval: chunking, indice, k, aggiornamento del corpus.

LA STESSA DOMANDA

senza RAG «Risultano 128 unità.»

con RAG «37 pz nel deposito B2.» [\[giacenze\\_B2.csv · 05/09/2026\]](#)

# Rendere visibile il processo: mostrare o nascondere i passaggi intermedi (lo switch)

## SWITCH ACCESO

Si vedono i chunk recuperati e il punteggio di similarità prima della risposta.

Serve a chi costruisce e a chi valuta: si capisce se un errore è di recupero o di generazione.

## SWITCH SPENTO

Solo la risposta finale: è l'esperienza dell'utente in produzione.

Comodo, ma indistinguibile da un modello che sta improvvisando.

- La trasparenza è una scelta di interfaccia: la pipeline è identica, cambia solo cosa si mostra.
- Per un sistema interno vale tenere i passaggi almeno nei log: sono la traccia di audit di ogni risposta.

# «Non lo so» come segnale di salute del sistema, non un difetto

- Un modello nudo non ha modo di dire «non lo so»: con il RAG quel confine diventa misurabile.
- Due leve: la soglia di similarità sotto la quale i chunk vengono scartati, e l'istruzione a non rispondere senza fonti.
- In un contesto aziendale un rifiuto è recuperabile; un numero inventato entra in un ordine.

DOMANDA FUORI PERIMETRO

«Qual è il fatturato del terzo trimestre?»

best match: 0.18 < soglia 0.35

«Nei documenti disponibili non trovo questo dato.»

# Oltre il RAG semplice: Knowledge Graph e Time-Aware RAG (accenno)

## KNOWLEDGE GRAPH

Oltre ai frammenti si indicizzano le relazioni fra entità: fornitore → articolo → commessa.

Risponde a domande che il top-k non copre: «quali commesse dipendono da questo fornitore?».

## TIME-AWARE RAG

Ogni frammento porta una validità temporale: la revisione vigente vince su quelle superate.

Senza questo, procedura 2019 e procedura 2026 competono con lo stesso punteggio.

Entrambi restano varianti dello stesso schema: scegliere bene cosa entra nella finestra. Non servono al primo progetto, servono quando il corpus cresce.

# Verso il modulo 4: da leggere dati esterni ad agire su di essi

## FIN QUI

Sola lettura: il sistema recupera testo e risponde.  
Nessun effetto sui sistemi aziendali.

## DAL MODULO 4

Scrittura: creare un ordine, aprire un ticket,  
aggiornare una giacenza.

- Una risposta sbagliata si corregge; un'azione sbagliata resta nel sistema.
- Cambia il modello di rischio: servono permessi, conferme e tracciabilità di ogni chiamata.
- Modulo 4 — Dalla chat all'agente: strumenti, decisioni e cosa succede quando il modello ne sceglie uno.